DAY-04

Q. How java stores Negative and Floating point Numbers?

① Negative Number.

byte b = -42;.

byte — — — — — — — —
(8 bits)

(-128 to 127)

① +42

0	0	1	0	1	0	1	0
128	64	32	16	8	4	2	1

00101010 = 42

⇒ byte b = 42;
└ 00101010 → [00101010] (b)

System.out.println(b);
= 42

② byte b = -42

use 2's Complement.

① 42.
00101010

② 1's complement → 11010101

③ Add +1 at LSB — 11010101 + 1 = 11010110 — 2's complement of -42.

⇒ byte b = -42
└ 11010110 → [11010110] (b)

System.out.println(b);

Q. How Computer understand the Number stored in Memory is Negative

① - Check MSB (most significant Bit) of Binary No.

Eg. 11010110

MSB → 1 → Negative No.
MSB = 0 → positive Number

MSB — LSB (least significant bit)

↑ It is reserved for sign bit

byte 0 1 1 1 1 1 1 1
128 64 32 16 8 4 2 1

biggest No exp +ve 127

→ (-128 to 127)

b = 11010110

System.out.println(b);

① 1's complement → 00101001

+ 1

② 2's complement → (00101010)$_2$

↓

$(42)_{10}$

So the Number is -42
it print this Numba.

Q. Why we do 2's complement instead of 1's complement?

Ans To handle 0 case. as.

b = 0 0 0 0 0 0 0 0

byte b = 0;
byte b = -0;

1's → 1 1 1 1 1 1 1 1 — store it in Memory.

MSB +1

2's → 1 0 0 0 0 0 0 0

Extra

00000000 → 0

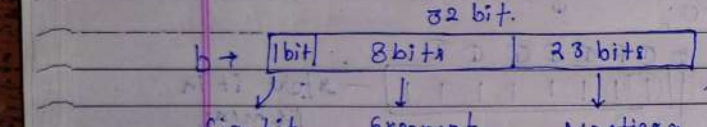
∴

-0 = 0

* * This prove that -0 ⊗ doesn't exist (-0 = 0)
2's complement handle this ~~dobly~~ doubly nature of 0

② Floating Number.

① float b = 8.125f; ② float f = 0.7f;



① b = 8.125

① $(8)_{10} \rightarrow (1000)_2$

② 0.125

0.125 * 2 = 0.25 → 0
0.25 x 2 = 0.5 → 0
0.5 x 2 = 1.0 → 1
0.0 x 2 → 0 x

record

① → $(1000.001)_2$

② Make it in the form of

$(1.xx) \times 2^{exp}$

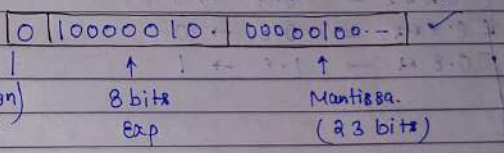
⇒ $1.000001 * 2^3$

③ Add Bias to the exponent.

for float bias = 127

exp ⇒ 3 + 127 ⇒ $(130)_{10} \rightarrow (10000010)_2$

④ place values in Memory.



→ System.out.println(b);

$(-1)^{sign} \times (1 + Mantissa) \times 2^{Exp - Bias}$

$(-1)^0 \times (1 + .000001) \times 2^{130-127}$

$1 \times \left(1 + \frac{1}{2^6}\right) \times 2^3$

$1 + \frac{1}{64} \times 8$

⇒ (1 + 0.015625) x 8

⇒ 8.125

② float f = 0.7f;

① 0.7

0.7 × 2 → 1.4 → 1
0.4 × 2 = 0.8 → 0
0.8 × 2 = 1.6 → 1
0.6 × 2 = 1.2 → 1
0.2 × 2 = 0.4 → 0
0.4 × 2 = 0.8 → 0
0.8 × 2 → 1.6 → 1

pattern repeat

① 0.7
↓ ↓
0. 0110 0110 0110 — —

② $1.xx \times 2^{exp}$

$1.0110\,0110\,\ldots \times 2^{-1}$

③ Add Bias.
$-1 + 127 \Rightarrow (126)_{10} \rightarrow (01111110)_2$

④ Express it in Memory.

0	01111110	0110 0110 0110	0110 0110 0110 0110 0110

1 bit ← 8 bit → ← 23 bit →

System.out.println(f);

$(-1)^{sign} \times (1 + \text{mantisa}) \times 2^{Exp-Bias}$.

$(-1)^0 \times \left(1 + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^6} + \frac{1}{2^7} + \frac{1}{2^{10}} + \frac{1}{2^{11}} + \frac{1}{2^{14}} + \frac{1}{2^{15}} + \frac{1}{2^{18}} + \frac{1}{2^{19}} + \frac{1}{2^{22}} + \frac{1}{2^{23}}\right) \times 2^{126-127}$

$\left(1 + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^6} + \frac{1}{2^7} + \frac{1}{2^{10}} + \frac{1}{2^{11}} + \frac{1}{2^{14}} + \frac{1}{2^{15}} + \frac{1}{2^{18}} + \frac{1}{2^{19}} + \frac{1}{2^{22}} + \frac{1}{2^{23}}\right) \times \frac{1}{2}$

≈ 0.69999998807907104421875.

≈ 0.7 (× 0.7)

That's why this happens in code.

① Bias. ?? Why Bias = 127 ??

S	exp	Mantissa
1 bit	8 bit	23 bit.

32 bits → IEEE format give it.

Add Bias to Neglect the Negative exponent. make it positive.

exp = 8 bits → $(0 - 255) + 2^8 \rightarrow 256$

↓

127

make Number unsigned to make calculation easy for processor.

$(2^{8-1} - 1) \Rightarrow (2^7 - 1) \rightarrow 127$ ✓

$(2^{exp-1} - 1)$ → Bias. formula

double d = 32.4186;

	exp	Mantissa
1 bit	11 bit	52 bits.

← 64 bits →

② Bias. for double

$(2^{11-1} - 1) = (2^{10} - 1) = 1023$ Ans

float → 0.7

double

→ Big decimal (concept of java.

0.7 — 0.7.